

Resumen teórico

Testing de Software — TP1

Índice

Introducción	3
1. Calidad de software, confiabilidad y motivación del testing	3
1.1. Por qué testear software	3
1.2. Calidad, confiabilidad y la naturaleza particular de las fallas de software .	4
1.3. Verificación y Validación (V&V)	4
1.4. Especificaciones, contratos e invariantes	5
2. Análisis estático y análisis dinámico	6
2.1. Análisis dinámico	6
2.2. Análisis estático	6
2.3. Una observación clásica de Dijkstra	7
3. Terminología fundamental: fault, error, failure, bug	7
3.1. Definiciones (capítulo 1, Ammann & Offutt)	7
3.2. Analogía médica	7
3.3. Ejemplo concreto: <code>numZero</code>	8
4. El modelo RIPR	8
4.1. Las cuatro condiciones	8
4.2. Por qué importa cada eslabón	9
5. Testing y debugging: dos procesos distintos	9
5.1. Definición de testing por niveles de pensamiento	10
6. Casos de prueba y cómo definirlos	10
6.1. Componentes de un test	10
6.2. Buenas prácticas al definir tests	11

6.3. Las dos preguntas centrales al diseñar tests	11
7. Procesos de desarrollo y rol del testing	12
7.1. Procesos clásicos	12
7.2. Procesos ágiles	12
7.3. Test-Driven Development (TDD)	12
7.4. Costo de detectar defectos tarde	12
8. Niveles de testing	12
8.1. Niveles tradicionales (modelo en V)	13
8.2. Testing de aceptación: usuario vs. negocio	13
8.3. Niveles específicos de programación orientada a objetos	13
8.4. Tipos básicos de testing por su objetivo	14
9. Criterios de cobertura y test requirements	14
9.1. El problema del espacio de entradas	14
9.2. Qué es un criterio de cobertura	14
9.3. Cuatro estructuras matemáticas, muchos criterios	15
10.Black-box, white-box y la perspectiva de MDTD	15
11.Actividades del testing y Model-Driven Test Design (MDTD)	16
11.1. Las cuatro actividades centrales	16
11.2. El proceso MDTD	17
11.3. Ejemplo mínimo de MDTD	17
12.Por qué testear: síntesis de objetivos	18
13.Síntesis final: ideas clave para fijar	18

Introducción

Este documento es un resumen teórico de las primeras unidades de *Testing de Software* de la UNRC. Está pensado como material de estudio para acompañar el TP1 y se apoya en dos fuentes complementarias: los capítulos 1 y 2 del libro *Introduction to Software Testing* (Ammann y Offutt, 2ª edición) y las notas de clase de la cátedra (notas 00, 01 y 02 de la docente Valeria Bengolea).

La idea es presentar los conceptos en un hilo lógico que vaya desde el problema general — por qué necesitamos testear software, qué relación tiene con la calidad y la confiabilidad — hasta el aparato técnico que la materia va a usar a lo largo del cuatrimestre: la distinción entre *fault*, *error* y *failure*, el modelo RIPR, la idea de criterios de cobertura y test requirements, los niveles y tipos de testing, y el proceso *Model-Driven Test Design* (MDTD) que articula todo. Más que copiar definiciones, intentamos reorganizarlas, conectarlas entre sí y agregar las intuiciones que las hacen comprensibles.

1. Calidad de software, confiabilidad y motivación del testing

1.1. Por qué testeamos software

El testing existe porque construir software es una actividad de ingeniería sujeta a errores humanos, y el software que producimos termina sosteniendo aspectos críticos de la vida cotidiana: redes, energía, transporte, dispositivos médicos, sistemas embebidos, banca, comercio electrónico. Ammann y Offutt abren el capítulo 1 con esta observación: a diferencia de otras industrias, el software está en todas partes y, al mismo tiempo, casi siempre se asume que “no va a fallar”. Esa asimetría entre la complejidad real y la expectativa optimista del usuario es lo que hace tan caro un defecto: cuando aparece, la consecuencia ya es visible para alguien.

Las notas refuerzan esta motivación con casos clásicos:

- **Therac-25:** una condición de carrera en el control de un acelerador médico provocó dosis letales de radiación.
- **Ariane 5:** una conversión de punto flotante de 64 bits a entero de 16 bits, perfectamente correcta para Ariane 4, provocó la pérdida del cohete a los 37 segundos del lanzamiento. El defecto era “de integración”: el software heredado nunca había sido reanalizado para la nueva trayectoria.
- **Mars Climate Orbiter:** un módulo trabajaba con unidades métricas y otro con unidades imperiales; la sonda se perdió al entrar a Marte.

- **Bug del Pentium (1994):** una tabla mal inicializada en la circuitería de división de punto flotante. El análisis posterior mostró que casi cualquier criterio de cobertura unitario lo hubiera detectado.
- **Zune:** un bucle de cálculo del año actual con la regla de los años bisiestos se quedaba en *loop* infinito durante el 31 de diciembre de un año bisiesto.

El reporte del NIST de 2002 citado tanto en el libro como en las notas pone una cifra concreta: un testing inadecuado cuesta a la economía estadounidense entre 22.000 y 59.000 millones de dólares anuales, y mejoras razonables en QA podrían reducir esa cifra a la mitad. Las fallas en aplicaciones financieras se miden directamente en millones por hora. La conclusión es directa: el testing no es un lujo de fin de proyecto, es una inversión.

1.2. Calidad, confiabilidad y la naturaleza particular de las fallas de software

La calidad del software se define a través de varios atributos: confiabilidad, mantenibilidad, interoperabilidad, seguridad, eficiencia, usabilidad, entre otros. De todos ellos la **confiabilidad** es uno de los más importantes y el más vinculado al testing. Una medida tradicional, aunque limitada, es la cantidad de defectos por línea de código: a mayor densidad de defectos, menos confiable es el producto.

Hay un punto sutil pero fundamental: las fallas de software **no son como las fallas mecánicas o eléctricas**. Una pieza mecánica falla por desgaste; un programa, no. El defecto que produce una falla de software está presente desde el momento mismo en que se escribió el código (o se diseñó el sistema, o se redactó el requisito) y solo se manifiesta cuando un escenario particular lo activa. Esto cambia la forma de pensar: no buscamos prevenir un deterioro, buscamos descubrir un error de diseño que ya está ahí.

1.3. Verificación y Validación (V&V)

La actividad general que abarca al testing se denomina **V&V**, y conviene distinguir sus dos componentes:

- **Verificación:** “¿Estamos construyendo el producto correctamente?”. Es decir, ¿el software cumple con la especificación que tenemos escrita?
- **Validación:** “¿Estamos construyendo el producto correcto?”. Es decir, ¿la especificación misma refleja lo que el usuario realmente necesita?

Las definiciones formales del libro lo dicen así (Definiciones 1.4 y 1.5): verificación es el proceso de determinar si los productos de una fase del desarrollo satisfacen los requisitos establecidos en la fase previa; validación es el proceso de evaluar el software al final del

desarrollo para asegurar que cumple con el uso pretendido. La verificación es más técnica y se apoya en especificaciones y requisitos formales; la validación depende del conocimiento del dominio.

Una intuición útil que aparece en las notas es ver el flujo

$$\textit{requisitos reales} \longrightarrow \textit{especificación} \longrightarrow \textit{implementación}$$

como dos saltos distintos: la validación atraviesa todo el flujo y pregunta si el resultado refleja los requisitos reales; la verificación se concentra en el último salto y pregunta si la implementación cumple con la especificación.

El problema general de la verificación se suele escribir de manera muy compacta como

$$Sys \models Spec$$

es decir: el sistema se comporta de acuerdo a su especificación. Operacionalmente, para programas imperativos, una manera muy útil de expresar esa especificación es mediante aserciones de corrección $\{P\} a \{Q\}$: siempre que el programa a comience en un estado que satisface la precondition P , debe terminar en un estado que satisface la postcondición Q .

1.4. Especificaciones, contratos e invariantes

Las especificaciones son fundamentales para razonar sobre la corrección y, por lo tanto, para diseñar tests con criterio. En las clases se trabajan tres niveles de formalidad:

- **Informales:** comentarios en lenguaje natural.
- **Semi-informales:** tags estilo Javadoc con `@precondition` y `@postcondition` escritas en prosa controlada.
- **Formales:** lenguajes como JML, que producen especificaciones *chequeables* mecánicamente.

Las precondiciones y postcondiciones establecen **contratos** entre cliente y proveedor con derechos y obligaciones simétricas:

- El **cliente** *debe* garantizar la precondition antes de llamar a la rutina y *puede* suponer la postcondición al regreso.
- El **proveedor** *puede* suponer la precondition al recibir el control y *debe* garantizar la postcondición al terminar.

Cuando una aserción se viola en tiempo de ejecución, el responsable depende de cuál se violó: si fue la precondition, el error es del cliente; si fue la postcondición, del proveedor.

Esto convierte a las aserciones en un mecanismo de *trazabilidad* del defecto.

A nivel de clases, además de pre/postcondiciones de cada método, se utilizan **invariantes de representación** (también llamados invariantes de clase): propiedades globales que toda instancia debe satisfacer. Las reglas son sencillas:

- Todos los constructores deben **establecer** el invariante al finalizar.
- Todos los métodos deben **preservarlo**: si el invariante valía antes de la llamada, debe seguir valiendo después.

Formalmente, esto se expresa pidiendo que cada método m satisfaga la tripla $\{\text{pre-}m \wedge \text{Inv}\} m \{\text{post-}m \wedge \text{Inv}\}$. En código, el invariante suele codificarse como un método `repOk()` que devuelve `true` si y solo si el objeto está bien formado; es una herramienta muy útil para testing y debugging.

2. Análisis estático y análisis dinámico

El proceso de V&V tiene dos grandes objetivos: *descubrir defectos* y *medir* si el sistema es usable en situaciones reales. Para lograrlo se combinan técnicas de **análisis** de los artefactos del software (programas, modelos, especificaciones, documentos). Para el código en particular, esas técnicas se agrupan en dos familias.

2.1. Análisis dinámico

Es el análisis que infiere propiedades del programa **ejecutándolo**. Tiene dos cualidades complementarias:

- Es **preciso**: lo que se observa efectivamente ocurrió en la corrida.
- Es **incompleto**: no cubre todos los comportamientos posibles, solo los inducidos por las entradas usadas.

Bajo este paraguas entran el *testing*, el *profiling* (consumo de recursos como memoria) y el chequeo en tiempo de ejecución de propiedades del programa.

2.2. Análisis estático

Infiere propiedades a partir del **texto** del programa, sin ejecutarlo. Es complementario al dinámico: en general puede cubrir todos los caminos posibles, pero a costa de ser **impreciso** (suele dar falsos positivos). Incluye inspección manual, chequeo de tipos, verificación estática de programas y herramientas de análisis. Una inspección manual cuidadosa puede ser sorprendentemente efectiva para encontrar defectos, aunque rara vez es exhaustiva.

2.3. Una observación clásica de Dijkstra

La frase más citada de la materia pertenece a Dijkstra (1970):

“Program testing can be used to show the presence of bugs, but never to show their absence.”

Esta cita captura el límite teórico del testing: aunque ejecutemos millones de tests, nunca probamos formalmente que no haya defectos. Lo que sí podemos hacer es *reducir el riesgo* y *aumentar la confianza*, y para eso necesitamos un proceso planificado, criterios objetivos y herramientas que cubran sistemáticamente el espacio de comportamientos del programa.

3. Terminología fundamental: fault, error, failure, bug

Una de las herramientas más útiles del curso es el vocabulario preciso para hablar de defectos. Conviene fijarlo desde el principio porque, en el habla cotidiana, “bug” se usa de manera ambigua para tres cosas distintas.

3.1. Definiciones (capítulo 1, Ammann & Offutt)

- **Fault (defecto):** “A static defect in the software”. Es el problema estático que vive en el código, el diseño o la especificación. Existe se ejecute o no el programa.
- **Error:** “An incorrect internal state that is the manifestation of some fault”. Es un estado interno incorrecto que aparece cuando el defecto se activa durante la ejecución.
- **Failure (falla):** “External, incorrect behavior with respect to the requirements or another description of the expected behavior”. Es la manifestación externa observable del problema.

La cadena causal habitual es

$$fault \longrightarrow error \longrightarrow failure.$$

Un **bug** es un término informal que en el lenguaje cotidiano agrupa a los tres. En el contexto de la materia preferimos los términos específicos.

3.2. Analogía médica

Para fijar las ideas, el libro y las notas usan una analogía con la medicina: un paciente llega al médico con una lista de *síntomas* (esos son las *failures*); el médico intenta diagnosticar la causa (el *fault*); para hacerlo puede medir condiciones anómalas internas como presión, arritmias o niveles de glucosa (esos son los *errors*). La analogía aclara la jerarquía, pero

hay una diferencia crucial: los fallos de software no provienen de un “desgaste natural” como muchas enfermedades, sino de **errores de diseño** humanos que estuvieron presentes desde el comienzo.

3.3. Ejemplo concreto: numZero

El libro presenta un método Java muy simple para ilustrar la diferencia entre los tres conceptos:

```
1 public static int numZero(int[] x) {  
2     int count = 0;  
3     for (int i = 1; i < x.length; i++) { // i = 1 está mal  
4         if (x[i] == 0) count++;  
5     }  
6     return count;  
7 }
```

El defecto (*fault*) es comenzar el recorrido en $i = 1$ en lugar de $i = 0$.

- Con $x = [2, 7, 0]$ el método devuelve 1, que es el resultado correcto. El defecto se ejecuta y produce un **error** (el valor de i es incorrecto en la primera iteración), pero el error *no propaga* a la salida y **no hay falla observable**.
- Con $x = [0, 7, 2]$ el método devuelve 0 en lugar de 1. Acá el error sí propaga a la variable `count`, y se traduce en una **falla** visible.

Este ejemplo muestra algo crucial: **ejecutar el defecto no garantiza ver la falla**. Para que la falla se manifieste se tienen que dar varias condiciones, que el modelo RIPR formaliza.

4. El modelo RIPR

RIPR — por **Reachability, Infection, Propagation, Revealability** — es el modelo central que la materia usa para razonar sobre cuándo y por qué un test revela un defecto. Aparece en el capítulo 2 del libro y se discute en la nota 02 con un diagrama que ilustra el flujo.

4.1. Las cuatro condiciones

Para que un test logre revelar un defecto deben darse, en orden, cuatro condiciones:

1. **Reachability (alcanzar)**: la entrada del test tiene que llevar la ejecución hasta el lugar (o lugares) del programa donde está el defecto.

2. **Infection (infectar)**: una vez ejecutado el código defectuoso, el estado del programa tiene que volverse incorrecto.
3. **Propagation (propagar)**: el estado infectado tiene que seguir vivo y propagarse a través del resto de la ejecución hasta provocar que alguna salida o estado final sea incorrecto.
4. **Revealability (revelar)**: el tester tiene que efectivamente *observar* la porción incorrecta del estado final, comparándola contra un oráculo. Si solo observa partes correctas, la falla queda oculta.

4.2. Por qué importa cada eslabón

RIPR explica por qué tantos defectos sobreviven al testing aunque parezca exhaustivo:

- Si no hay *reachability*, el código defectuoso nunca se ejecuta (por ejemplo, una condición que solo se da en producción).
- Si no hay *infection*, el código se ejecuta pero produce el mismo valor que la versión correcta para esa entrada (caso `numZero([2,7,0])`).
- Si no hay *propagation*, el estado interno está mal pero se sobrescribe antes de salir.
- Si no hay *revealability*, la falla está en la salida pero el oráculo no la mira.

Este modelo aplica incluso cuando el defecto es por *omisión*: cuando falta código que debería estar, el contador de programa al pasar por esa zona “ausente” ya constituye un estado erróneo. Diseñar tests con RIPR en mente es lo que diferencia un test que “llama al método” de uno que efectivamente busca exponer un defecto.

5. Testing y debugging: dos procesos distintos

Es habitual confundir *testing* con *debugging*; las notas y el libro los separan con cuidado.

- **Testing**: “Evaluating software by observing its execution”. Es el proceso de *evaluar* el comportamiento del software ejecutándolo. El resultado de un test es “pasa / falla”.
- **Test Failure**: ejecución de un test que da lugar a una falla en el software.
- **Debugging**: “The process of finding a fault given a failure”. Es lo que viene *después* de un test que falla: a partir de la falla observada, buscamos el defecto en el código que la causó.

Un test puede ser *exitoso* (no revela falla) o *fallido* (descubre una falla). En testing, paradójicamente, un test “fallido” es *información valiosa*: nos dice dónde mirar.

5.1. Definición de testing por niveles de pensamiento

Beizer propuso una caracterización del testing en cinco *niveles de madurez* según cómo lo entiende cada profesional. El libro y las notas la adoptan:

- **Nivel 0:** no hay diferencia entre testing y debugging. Es la postura ingenua del estudiante de programación: “hago compilar, pruebo con un par de entradas, corrijo lo que sale mal”.
- **Nivel 1:** el propósito del testing es mostrar **corrección**. Es un paso adelante, pero teóricamente imposible: la corrección no se puede establecer ejecutando un número finito de tests.
- **Nivel 2:** el propósito del testing es mostrar que **el software no funciona**. Es un objetivo válido (encontrar fallas), pero pone a testers y desarrolladores en relación adversaria.
- **Nivel 3:** el propósito no es probar nada específico, sino **reducir el riesgo** asociado al uso del software. Acepta que toda decisión de usar software implica algún riesgo y orienta el trabajo a controlarlo.
- **Nivel 4:** testing es una **disciplina mental** que ayuda a los profesionales a desarrollar software de mejor calidad. La responsabilidad principal no es solo medir, sino *mejorar* la calidad.

Esta escala es útil porque obliga a preguntarse en qué nivel está la organización (o uno mismo) y qué cambios harían falta para pasar al siguiente.

6. Casos de prueba y cómo definirlos

6.1. Componentes de un test

Un test —para ser útil— no es solo una entrada. Las notas lo descomponen en tres partes mínimas:

1. **Entradas:** los valores que se pasan al código bajo prueba.
2. **Código a ejecutar (SUT, *Software Under Test*):** el método o componente que se testea.
3. **Descripción de los resultados esperados (condición de aceptación):** el *oráculo*, contra el cual se contrasta lo que realmente devolvió el programa.

En la práctica de JUnit esto se materializa así:

```
1 @Test
```

```
2 void oddOrPosTest() {  
3     int[] x = {-3, -2, 0, 1, 4};  
4     int result = SimpleRoutines.oddOrPos(x);  
5     assertEquals(3, result, "Array contains positive number");  
6 }
```

La elección de la entrada `{-3,-2,0,1,4}` no es casual —tiene un impar negativo, un par negativo, un cero, un impar positivo y un par positivo— y la aserción `assertEquals(3, result)` actúa como oráculo.

6.2. Buenas prácticas al definir tests

Las notas (siguiendo a Myers y otros) listan varios principios:

- Un programador debería evitar testear sus propios programas: psicológicamente no estamos preparados para buscar errores en nuestro propio trabajo, y un error frecuente es que el programador no entendió bien el enunciado o la especificación, en cuyo caso testeará el mismo malentendido.
- Esto está cambiando con metodologías ágiles y *Test-Driven Development*, donde escribir tests es parte natural del trabajo del desarrollador.
- Probar que el software *no haga lo que debe hacer* es solo la mitad del trabajo: la otra mitad es probar que *no haga lo que no debe hacer*.
- Evitar los **tests desechables**: si tuviéramos que reinventar los tests cada vez que volvemos al programa, perdemos todo el trabajo. Guardar tests y volver a ejecutarlos tras cambios en otras componentes es lo que se llama **regression testing** (testing de regresión).

6.3. Las dos preguntas centrales al diseñar tests

Las notas formulan dos preguntas que vuelven a aparecer una y otra vez en la materia:

- ¿Cómo **elegimos** las entradas?
- ¿Contra qué **contrastamos** el resultado obtenido (oráculo)?

El resto de la materia se dedica a dar respuestas progresivamente más sofisticadas a estas dos preguntas.

7. Procesos de desarrollo y rol del testing

7.1. Procesos clásicos

Los modelos clásicos de proceso de desarrollo (estilo cascada) involucran fases relativamente bien separadas: requisitos, especificación, planeamiento, diseño, implementación, integración y testing, mantenimiento. En estos modelos, el testing aparece principalmente como una fase *posterior a la implementación*.

7.2. Procesos ágiles

En los modelos ágiles el testing está presente en varias fases del proceso, principalmente como parte de la codificación misma. Cada nueva funcionalidad se acompaña de tests; el ciclo típico es escribir código → mejorar tests → ejecutar tests → reparar si hay errores → repetir hasta cubrir la especificación.

7.3. Test-Driven Development (TDD)

En TDD se da una vuelta de tuerca adicional: los **tests se escriben antes de la codificación** y funcionan como *especificación de la funcionalidad* y *criterio de aceptación* del código. El ciclo es: escribir tests → escribir código para que pasen → ejecutar tests → reparar / mejorar → repetir.

7.4. Costo de detectar defectos tarde

Una idea muy importante del capítulo 1 (con la famosa Figura 1.1) es que el **costo** de encontrar y reparar un defecto crece dramáticamente cuanto más tarde se lo detecta:

- Detectarlo durante requisitos, diseño o testing unitario: costo unitario.
- Durante integración: $\sim 5\times$.
- Durante system testing: $\sim 10\times$.
- Después del despliegue: $\sim 50\times$.

Por eso la consigna estratégica de Beizer (nivel 4) se condensa en: “eliminar defectos tan temprano como sea posible”. Cada falla atrapada en testing unitario ahorra dinero.

8. Niveles de testing

Los niveles de testing son una categorización clásica, frecuentemente representada con el **modelo en V**, que asocia a cada fase del desarrollo un nivel de testing.

8.1. Niveles tradicionales (modelo en V)

- **Acceptance testing:** evalúa el software respecto a los requisitos y necesidades del usuario. ¿Hace el software lo que el cliente quería?
- **System testing:** evalúa el sistema completo contra su diseño arquitectónico y su comportamiento global. Suele encontrar fallas costosas porque ya hay integración entre módulos.
- **Integration testing:** evalúa cómo se comunican entre sí los módulos. Asume que cada módulo funciona bien individualmente y se concentra en las *interfaces*.
- **Module testing:** evalúa cada módulo (típicamente una clase o un paquete) en aislamiento, incluyendo cómo interactúan internamente sus unidades.
- **Unit testing:** evalúa la unidad más chica (un método o procedimiento) de manera aislada. Es el nivel “más bajo”.

8.2. Testing de aceptación: usuario vs. negocio

El testing de aceptación se subdivide a su vez en dos:

- **Aceptación de usuario:** lo realiza el usuario para determinar si el sistema cubre sus necesidades. Es esencialmente *validación*.
- **Aceptación de negocio:** lo realiza el equipo de desarrollo/testing para determinar si el sistema eventualmente pasará las pruebas del usuario. Es esencialmente *verificación*.

8.3. Niveles específicos de programación orientada a objetos

El testing en orientación a objetos rompe un poco la dicotomía unit/module, porque las clases agrupan muchos métodos relacionados. Por eso se introducen niveles más finos:

- **Intra-method testing:** testear cada método individualmente.
- **Inter-method testing:** testear pares de métodos de la misma clase (cómo interactúan vía el estado del objeto).
- **Intra-class testing:** testear una clase entera mediante secuencias de llamadas a sus métodos.
- **Inter-class testing:** testear varias clases juntas; es una forma de testing de integración.

8.4. Tipos básicos de testing por su objetivo

Independientemente del nivel, las notas mencionan algunos tipos de testing que se distinguen por su *objetivo* o *condición de aceptación*:

- **Regression**: la condición de aceptación es que se preserve el comportamiento de versiones anteriores del software. Se guardan los tests y se vuelven a ejecutar tras cada cambio.
- **Diferencial**: se contrasta el comportamiento contra otro software con el mismo propósito.
- **Aceptación**: se evalúa contra los requisitos o necesidades del usuario.

9. Criterios de cobertura y test requirements

9.1. El problema del espacio de entradas

Aunque parezca que un método como

```
1 private static double computeAverage(int A, int B, int C)
```

es trivial, el espacio de entradas posibles es gigantesco: tres enteros de 32 bits dan $\sim 80 \cdot 10^{27}$ combinaciones distintas. A efectos prácticos, el espacio de inputs es **infinito**. Ejecutar “todos los tests” es inviable.

El tester se enfrenta entonces a dos preguntas estructurales:

1. ¿Cómo **buscamos** en ese espacio?
2. ¿Cuándo **paramos** de buscar?

9.2. Qué es un criterio de cobertura

Los **criterios de cobertura** son la herramienta principal para responder esas dos preguntas. Un criterio de cobertura es una regla (o un conjunto de reglas) que define un conjunto de *test requirements*: elementos específicos de un artefacto que cada test debe satisfacer o cubrir.

- **Criterio de test**: colección de reglas y un proceso que define los test requirements. Ejemplos: “cubrir cada línea de código”, “cubrir cada requisito funcional”.
- **Test requirement**: elemento específico de un artefacto de software que un test debe satisfacer. Cada línea de código (o cada rama, o cada par de aristas...) puede dar lugar a un test requirement.

Satisfacer un criterio de cobertura le da al tester dos garantías importantes:

- La búsqueda en el espacio de entradas se hizo de manera **minuciosa**: no quedan rincones obvios sin explorar.
- Los tests no tienen **demasiado solapamiento**: no estamos redundando trabajo.

Además, los criterios proveen una **regla de parada** (*stopping rule*): se sabe *a priori* cuántos tests son necesarios para satisfacer el criterio y se sabe cuándo se terminó.

9.3. Cuatro estructuras matemáticas, muchos criterios

Una observación profunda del libro de Ammann & Offutt, que organiza toda la segunda parte de la materia, es que los cientos de criterios de cobertura definidos en la literatura se pueden **reducir** a unos pocos sobre cuatro tipos de estructuras matemáticas:

1. **Dominio de entradas** (*input domain*): particiones del espacio de entradas.
2. **Grafos**: grafos de control de flujo, grafos de llamadas, máquinas de estado.
3. **Expresiones lógicas**: condiciones, decisiones, MC/DC.
4. **Descripciones sintácticas**: gramáticas, mutaciones del programa.

El criterio “cobertura de pares de aristas” en un grafo de control de flujo, por ejemplo, generará un test requirement por cada par de aristas consecutivas; el tester deberá encontrar caminos (y luego inputs concretos) que recorran todos esos pares.

10. Black-box, white-box y la perspectiva de MDTD

Una clasificación tradicional del testing es:

- **Testing de caja negra (black-box)**: los tests se generan a partir de descripciones *externas* del software — especificaciones, requisitos, diseño. No se mira el código.
- **Testing de caja blanca (white-box)**: los tests se generan a partir del *código* mismo, específicamente sus ramas, condiciones y líneas. Algunos autores agregan “gray-box” para tests que parten del diseño interno.

Hay una observación importante del libro: aunque históricamente se asoció black-box con testing de sistema y white-box con unit testing, esa correspondencia es *artificial*. Cualquiera de los dos enfoques se puede aplicar a cualquier nivel.

Más aún, desde la perspectiva de **MDTD** (Model-Driven Test Design) la distinción pierde importancia: lo que importa es **de qué nivel de abstracción se deriva el modelo** con

el que se diseña el test. Un grafo de control de flujo extraído del código y un grafo extraído de un diagrama UML se manejan con los mismos criterios; lo que cambia es la fuente.

11. Actividades del testing y Model-Driven Test Design (MDTD)

11.1. Las cuatro actividades centrales

El capítulo 2 separa el trabajo de testing en cuatro actividades discretas, cada una con habilidades, conocimientos y formación distintos:

1. **Diseño de tests (test design)**: diseñar valores de entrada que efectivamente ejerciten el software. Se subdivide en dos sub-enfoques:
 - **Basado en criterios**: diseñar tests para satisfacer un criterio de cobertura u otro objetivo formal. Es la parte más matemática del testing.
 - **Basado en factor humano** (*human-based* o *stress tests*): diseñar tests aprovechando el conocimiento del dominio y del comportamiento típico/extremo de los usuarios. Busca activamente entradas raras, valores límite, situaciones no previstas.
2. **Automatización de tests (test automation)**: codificar los tests en *scripts ejecutables* (por ejemplo, casos de JUnit). Requiere habilidad de programación.
3. **Ejecución de tests (test execution)**: correr los tests sobre el software y registrar los resultados. Si están bien automatizados es trivial.
4. **Evaluación de tests (test evaluation)**: evaluar los resultados, interpretarlos y reportarlos a desarrolladores o gerentes. Es mucho más difícil de lo que parece porque requiere conocimiento del dominio, de testing y de comunicación.

Además de estas cuatro, hay actividades de soporte: **gestión de tests** (organización del equipo, elección de criterios, grado de automatización), **mantenimiento de tests** (guardar y actualizar tests a medida que el software evoluciona) y **documentación de tests** (criterios y requisitos cubiertos, justificación de tests diseñados a mano).

Los enfoques basados en criterios y basados en humano **no compiten**, se complementan: lo cuantitativo (criterios formales) y lo cualitativo (intuición humana del dominio) cubren tipos de defectos distintos.

11.2. El proceso MDTD

MDTD propone un proceso explícito de diseño de tests que “eleva el nivel de abstracción”. La idea es exactamente la misma que en ingeniería tradicional: en lugar de razonar directamente sobre la implementación, se construye un **modelo matemático** (un grafo, una partición, una expresión lógica, una gramática), se trabaja a ese nivel y luego se baja a valores concretos.

El flujo, ilustrado en la figura 2.4 del libro y en las transparencias de la nota 02, se puede leer como una cadena de transformaciones:

1. **Artefacto software** (código, UML, requisitos, manual de usuario).
2. De ese artefacto se extrae un **Modelo / Estructura** (por ejemplo, un grafo de control de flujo).
3. Aplicando un criterio sobre ese modelo se generan **Test requirements**.
4. Los test requirements se refinan a **especificaciones de tests** (caminos concretos a recorrer, valores a probar).
5. Se eligen **Valores de inputs** que satisfacen las especificaciones.
6. Esos valores se empaquetan como **tests**.
7. Los tests se traducen a **scripts ejecutables**.
8. Se obtienen **resultados** y, finalmente,
9. se evalúa cada resultado como **pasa / falla**.

Lo de arriba se divide en dos planos: el plano de *diseño* (artefacto → modelo → requisitos → especificaciones) y el plano de *implementación* (valores → tests → scripts → resultados → evaluación). MDTD separa también las cuatro actividades del punto anterior: el diseño, la automatización, la ejecución y la evaluación quedan claramente delimitadas y pueden ser realizadas por personas distintas.

11.3. Ejemplo mínimo de MDTD

El propio capítulo 2 ilustra el proceso con un método Java muy simple:

```
1 /** Return index of node n at the first position it appears ,
2  *   -1 if it is not present */
3 public int indexOf(Node n) {
4     for (int i = 0; i < path.size(); i++)
5         if (path.get(i).equals(n))
6             return i;
```

```
7     return -1;
8 }
```

A partir del código se extrae un grafo de control de flujo con cinco nodos (inicio, guarda del `for`, evaluación del `if`, `return i`, `return -1`) y aristas que reflejan el flujo. Si se elige el criterio de **cobertura de pares de aristas**, se obtienen seis test requirements (subcaminos como `[1,2,3]`, `[1,2,5]`, `[2,3,4]`, etc.), y se buscan tres caminos completos que entre todos cubran los seis pares. Recién al final se eligen valores concretos de `path` y `n` que recorran esos caminos.

12. Por qué testeamos: síntesis de objetivos

A partir de todo lo anterior, las notas cierran la unidad de introducción con una lista compacta de **objetivos** del testing:

- Revelar fallas tan temprano como sea posible durante el desarrollo.
- **Mejorar la calidad** del software: si encontramos fallas, las corregimos antes del despliegue.
- **Reducir el costo** de mantenimiento y operación: cada falla atrapada en testing unitario evita costos 50× mayores tras despliegue.
- Mantener a los **usuarios satisfechos** y proteger la confianza en el producto.
- Más estratégicamente (nivel 3), **reducir el riesgo** asociado al uso del software.
- En el nivel más maduro (nivel 4), **aumentar la calidad** de manera sostenida formando a los desarrolladores: medir, reportar, formar, prevenir.

13. Síntesis final: ideas clave para fijar

Como cierre, las ideas más importantes que deberían quedar fijas después de leer este resumen son las siguientes:

1. **Testing no es debugging.** Testing evalúa el software para revelar fallas; debugging busca el defecto causal a partir de una falla observada. Son procesos distintos con habilidades distintas.
2. **Fault, error, failure** forman una cadena causal: el defecto vive en el código, se manifiesta como un estado interno incorrecto cuando se ejecuta, y produce una falla cuando ese estado infectado se propaga a una salida observable.

3. El **modelo RIPR (Reachability, Infection, Propagation, Reveability)** es el marco para diseñar tests que realmente expongan defectos: alcanzar la línea, infectar el estado, propagar el error a la salida y observarlo con un oráculo.
4. El **testing no prueba ausencia de defectos**, solo presencia (Dijkstra). Por eso se trabaja con criterios objetivos que sistematicen la exploración y provean reglas de parada.
5. **Verificación y validación** son distintas: la primera mira si construimos el producto correctamente respecto a la especificación; la segunda si construimos el producto correcto respecto a las necesidades reales.
6. Las **especificaciones** —informales, semi-informales o formales (JML)— son la base para razonar sobre corrección. Los **contratos** de pre/postcondiciones y los **invariantes de clase** estructuran ese razonamiento y permiten asignar responsabilidad ante cada violación.
7. Un **test** se compone de entradas, código a ejecutar y oráculo. Sin oráculo el test no sirve: nunca sabremos si lo que pasó está bien o mal.
8. Los **criterios de cobertura** son la respuesta estructurada al problema del espacio infinito de entradas: definen un conjunto finito de *test requirements* que organiza la búsqueda y dicta cuándo parar.
9. Todo criterio razonable se puede definir sobre **cuatro estructuras matemáticas**: dominio de entradas, grafos, expresiones lógicas y descripciones sintácticas. Esta es la columna vertebral de la materia.
10. Los **niveles de testing** (unit, module, integration, system, acceptance) y sus variantes para POO (intra-method, inter-method, intra-class, inter-class) describen *a qué nivel del software apunta el test, no cómo se diseña*.
11. La **automatización** (JUnit, scripts) es lo que vuelve viable el regression testing y, en última instancia, la capacidad de mantener un software vivo sin romperlo.
12. **MDTD (Model-Driven Test Design)** es el proceso global: del artefacto al modelo, del modelo a los requisitos, de los requisitos a valores concretos, de los valores a scripts y al veredicto pasa/falla. Separa cuatro actividades (diseño, automatización, ejecución, evaluación) que requieren perfiles distintos.
13. El testing es, en última instancia, una **disciplina de calidad**: detectar defectos temprano abarata, reduce el riesgo del uso del software y, en organizaciones maduras, mejora a quienes lo construyen.

Con este marco como base, las próximas unidades (cobertura de grafos, cobertura lógica, particionado del espacio de entradas, testing basado en sintaxis, property-based testing y generación automática) van a verse como aplicaciones concretas de las mismas ideas: un modelo, un criterio que produce test requirements, valores que los satisfacen y un oráculo que decide pasa/falla.